Exam — Databases - Summer 2026

Tasks A — Explain Database Terms

Explain the following five terms in 2-3 sentences each and give one concrete database example for each term.

- *Indexing*
- *Concurrency*
- *Join*
- *Aggregation*
- *Denormalization*

Tasks B — Multiple Choice: True or False

For each statement, mark exactly one column. For every correct answer, you receive 2 points. For every wrong answer, 2 points are deducted. If you leave a row empty or mark both columns, you receive 0 points for that row.

Statement	***True***	***False***
A table can have multiple primary keys.		
Denormalization always improves query performance.		
Indexing always reduces the storage space required for a database.		
A database in Third Normal Form (3NF) is automatically in Second Normal Form (2NF).		
Concurrent transactions can never cause data inconsistency if they only perform read operations.		
A candidate key can contain NULL values.		
The `UNION` operation in SQL automatically removes duplicate rows.		
A foreign key value may be NULL unless the column is declared NOT NULL.		

Tasks C — Normalization and SQL Constraints

A small event platform stores ticket purchases in one table:

```SQL
CREATE TABLE ticket_sales (
    sale_id INT,
    sale_date DATE,
    customer_id INT,
    customer_name VARCHAR(100),
    customer_email VARCHAR(100),
    event_id INT,
    event_title VARCHAR(150),
    event_date DATE,
    venue_id INT,
    venue_name VARCHAR(100),
    venue_city VARCHAR(100),
    ticket_type VARCHAR(50),
    ticket_price DECIMAL(8, 2),
    quantity INT,
    payment_method VARCHAR(50),
    PRIMARY KEY (sale_id, event_id, ticket_type)
);
```

1. Identify three functional dependencies that are visible in the table. Use the form — 6 P.

   ```SQL
   A -> B, C
   ```

   (`A` describes `B` and `C`)

2. Name two concrete anomalies or redundancies that can occur in this table. For each one, refer to specific columns from the schema. — 6 P.

3. Decompose the table into relations in 3NF. Write relation schemas with primary keys underlined using `(PK)` and foreign keys marked with `(FK)`. You do not need SQL syntax in this subtask. — 14 P.

4. Write SQL `CREATE TABLE` statements for two of your relations. Each statement must include a primary key and at least one useful constraint. At least one statement must include a foreign key. — 8 P.

Tasks D — Query Writing

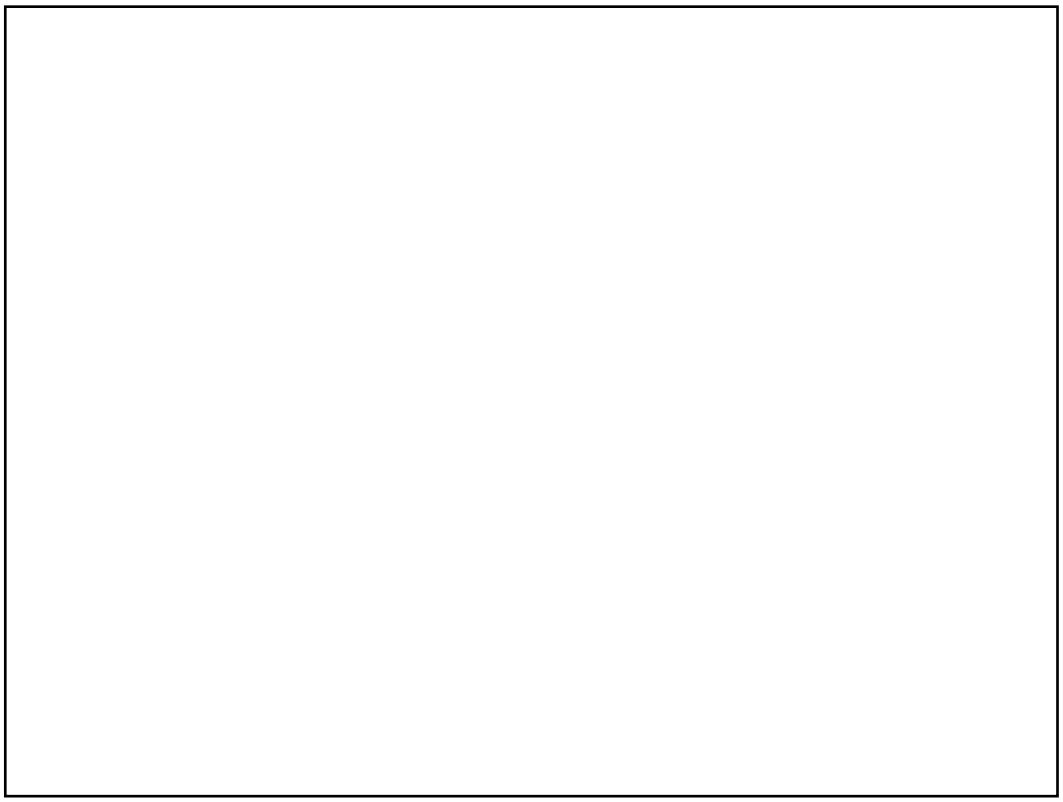
Use the following relational schema:

```sql
Customer(customer_id, name, email)
Product(product_id, name, category, list_price)
OrderHeader(order_id, customer_id, order_date, status)
OrderItem(order_id, product_id, quantity, unit_price)
```

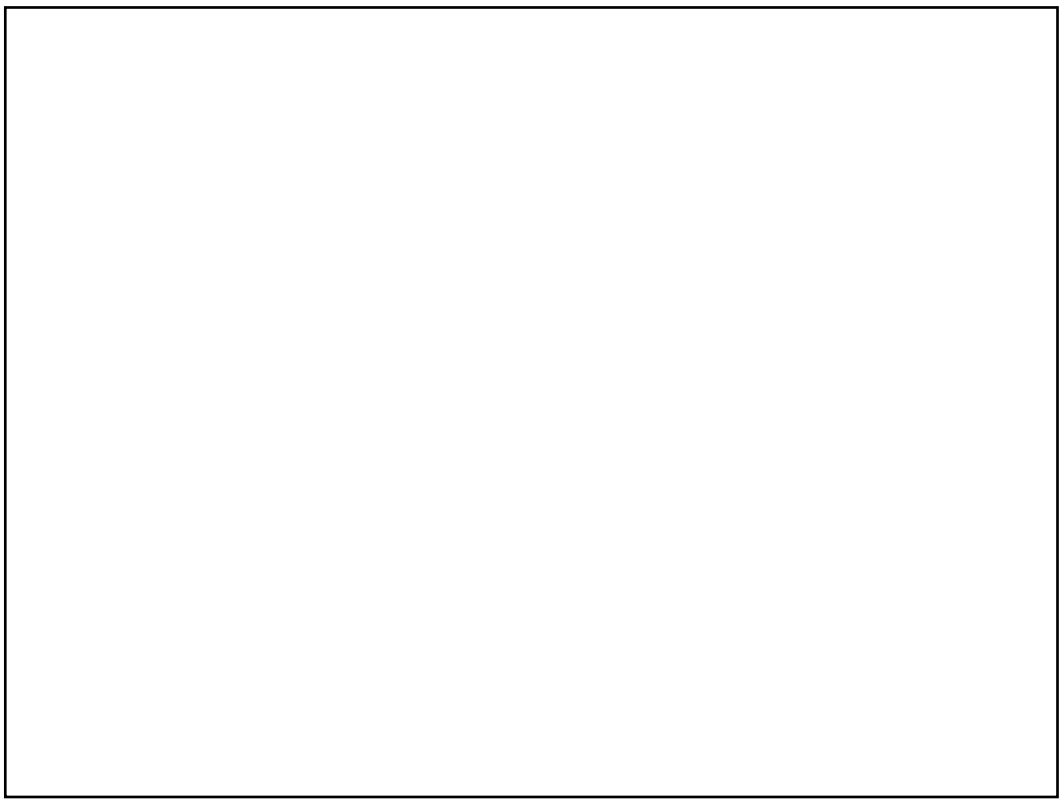
Primary keys are the first listed attributes, except `OrderItem`*, where* `(order_id, product_id)` *is the primary key.* `OrderHeader.customer_id` *references* `Customer.customer_id`*.* `OrderItem.order_id` *references* `OrderHeader.order_id`*.* `OrderItem.product_id` *references* `Product.product_id`*.*

1. Write a query that returns the names and emails of customers who have at least one order with status `shipped`. 6 P.

2. Write a query that returns the total revenue per product category. Revenue is `quantity * unit_price`. Return `category` and `revenue`. 8 P.



3. Write a query that returns all customers who have never placed an order. 8 P.



Tasks E — Optimized Design Task: Hospital Management System

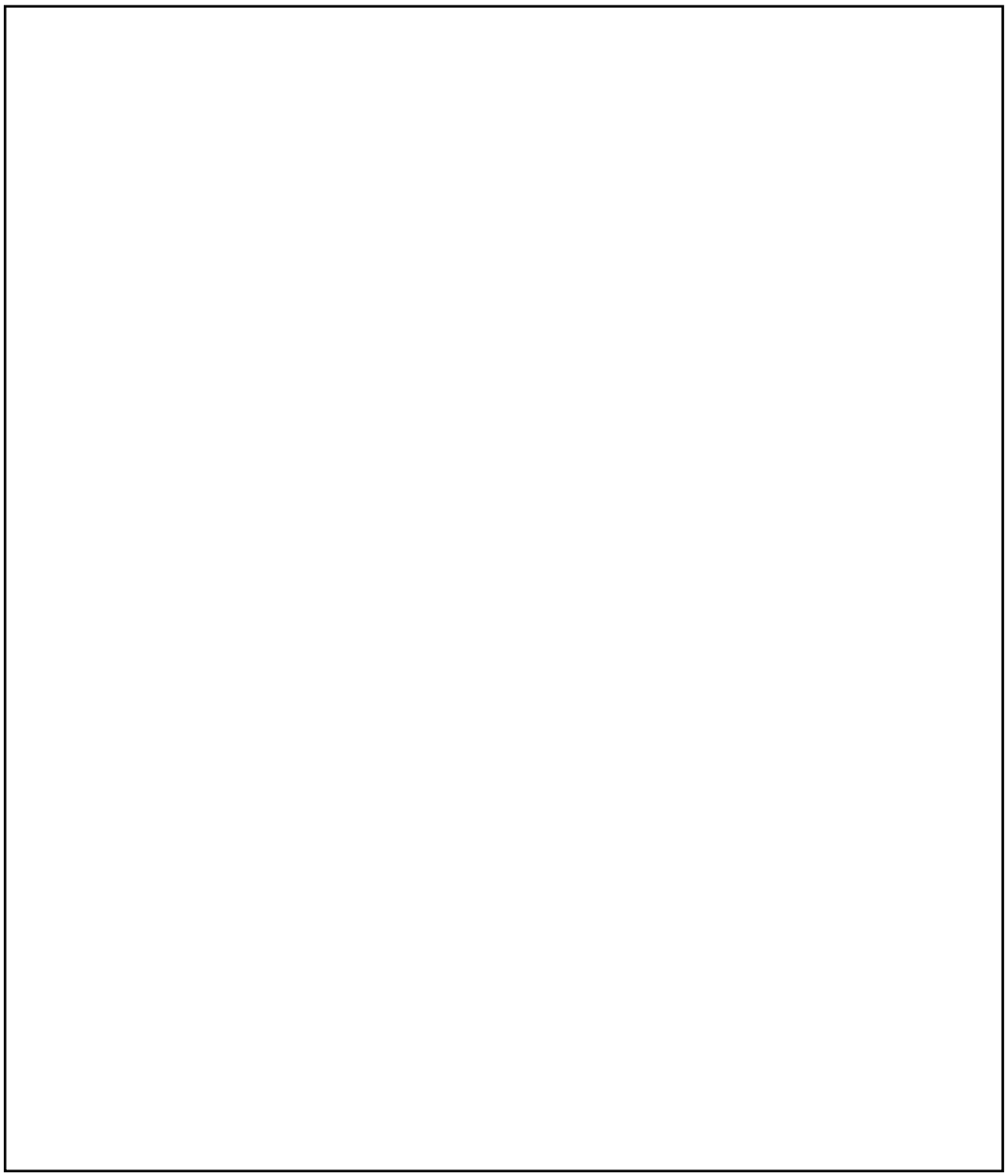
Design a database for the following restricted hospital scenario.

A hospital has departments. Each department has a unique name and employs doctors. Each doctor works for exactly one department and has a medical license number. Patients can be admitted to the hospital multiple times. Each admission has an admission date, an optional discharge date, and exactly one attending doctor. During an admission, doctors record treatments. A treatment has a timestamp, a treatment type, notes, and the doctor who performed it. The same admission can have many treatments. A patient has a name, date of birth, and insurance number. Insurance numbers are unique when known, but they may be missing.

Use surrogate integer IDs where useful. You may add attributes if needed, but the required information above must be represented.

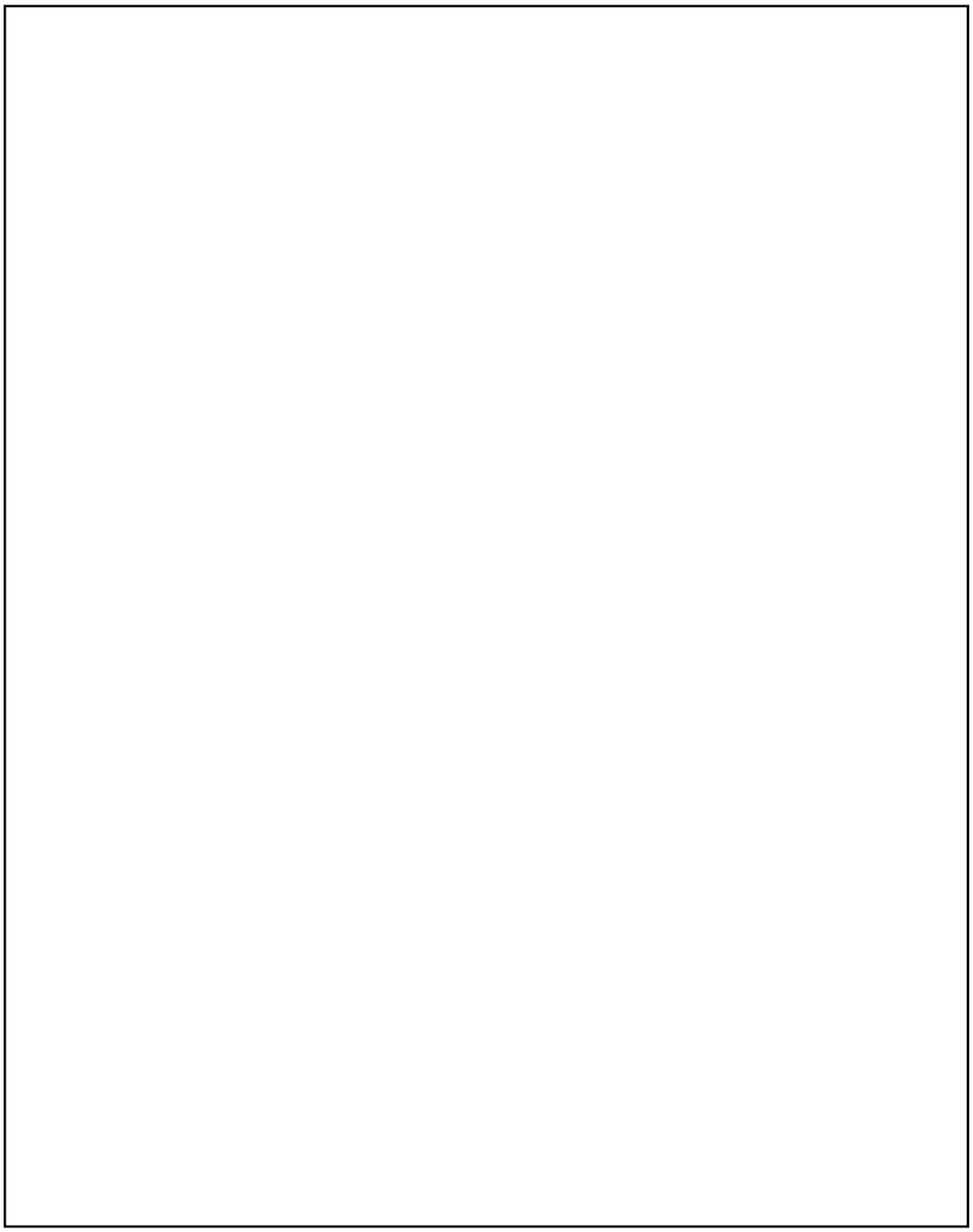
1. Draw or describe the ER model. Include the required entity types, relationships, and cardinalities. Write cardinalities explicitly if your drawing is ambiguous. 12 P.

   Grading checklist: entities (4 P), required attributes and identifiers (3 P), cardinalities (3 P), handling repeated admissions and treatments (2 P).



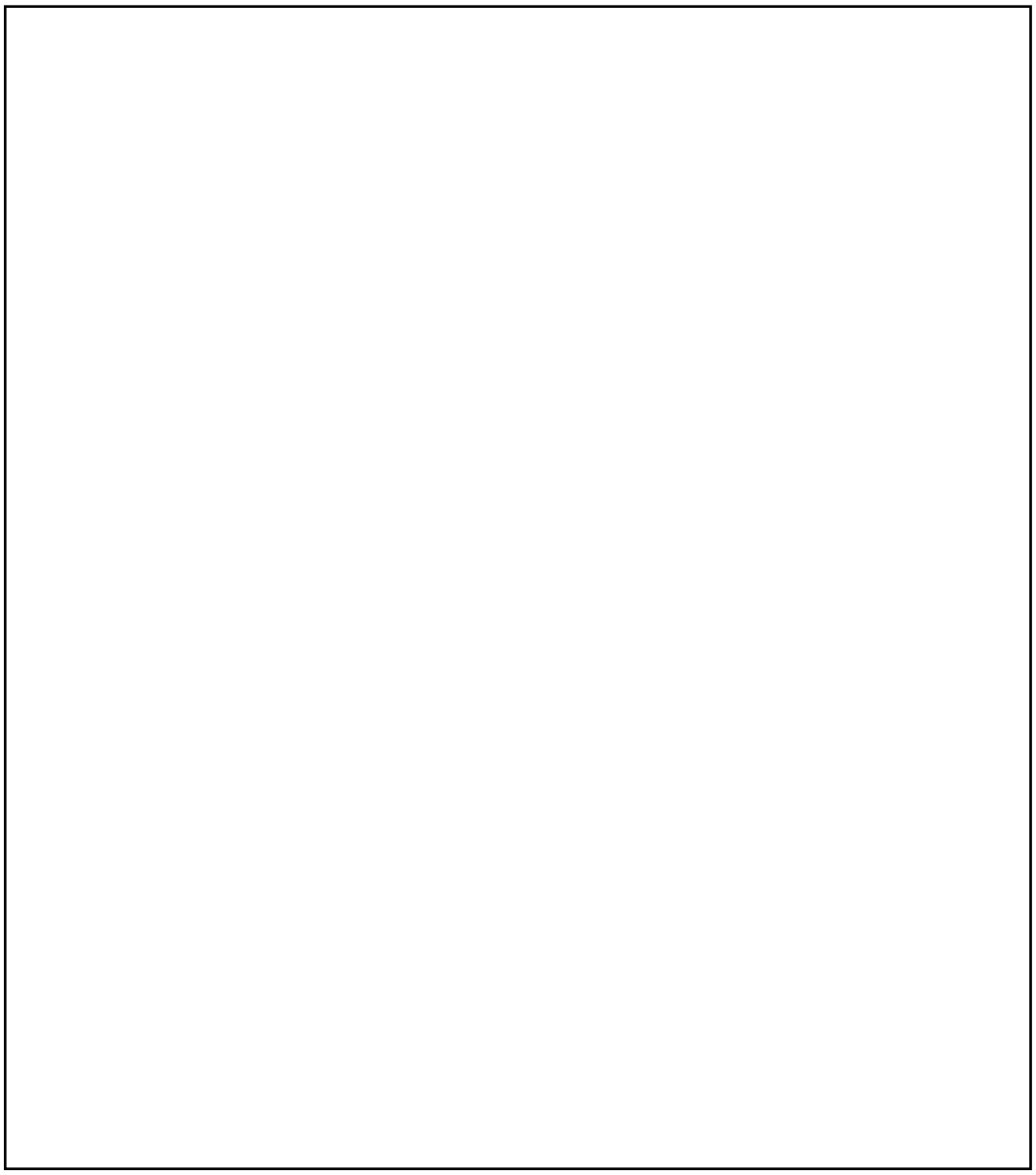
2. Transform your ER model into a relational model. Write every relation as `Table(attribute, attribute, ...)`, mark primary keys with `(PK)` and foreign keys with `(FK)`. 14 P.

   Grading checklist: correct tables (4 P), primary keys (3 P), foreign keys (4 P), optional values and uniqueness constraints (2 P), no unnecessary duplicated department/doctor/patient data (1 P).



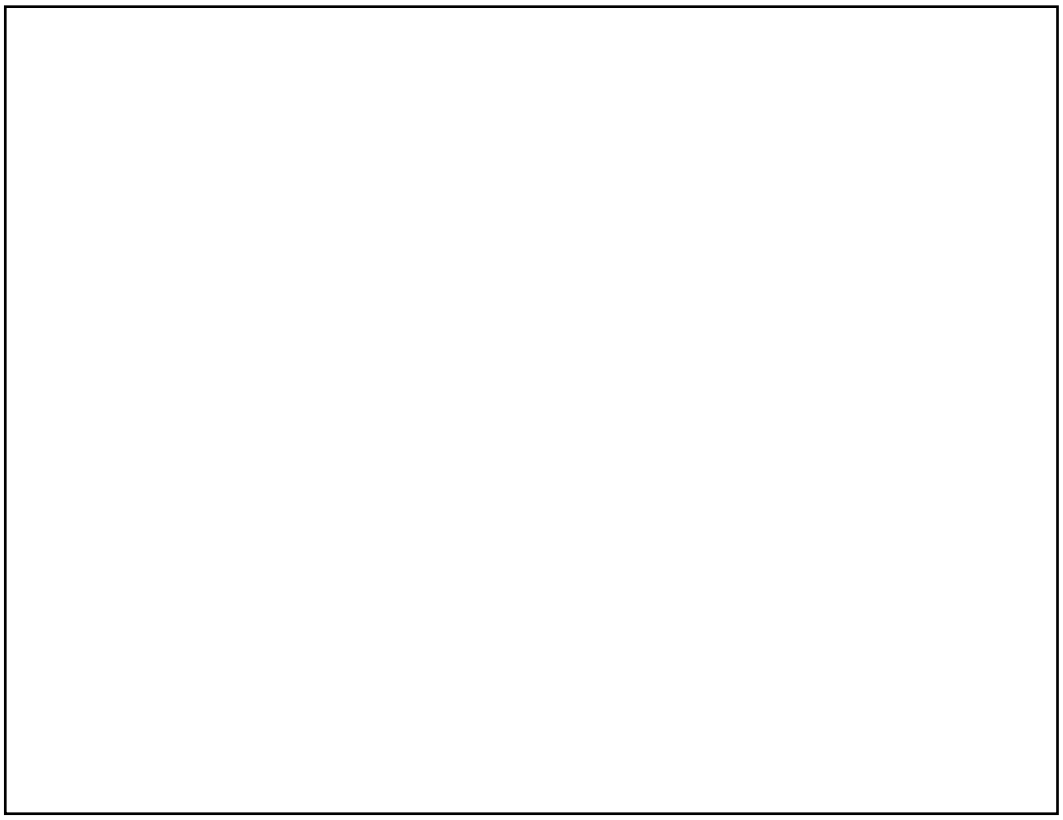
3. Write PostgreSQL `CREATE TABLE` statements for `Admission` and `Treatment`. Include primary keys, foreign keys, and constraints for dates or required values where appropriate. 10 P.

   Grading checklist: syntactically valid DDL (2 P), PKs (2 P), FKs (3 P), `NOT NULL`/ optional values handled correctly (2 P), useful date constraint (1 P).



4. Write a SQL query that returns all currently admitted patients with their attending doctor's name and department name. A patient is currently admitted if `discharge_date IS NULL`. 8 P.

   Grading checklist: correct joins (4 P), filter for current admissions (2 P), selected output columns and aliases are understandable (2 P).



In total, you can achieve 100 + 0 points. You have achieved _____ P. of 100 points.

Points	100–90	89–80	79–70	69–60	59–51	50–0
Value	A	B	C	D	E	F

Suggestion for the Solution – Exam

Task	Achieved Points
Tasks A — Explain Database Terms	_____ / **0**
Indexing: data structure that improves lookup speed at the cost of storage and write overhead. Example: B-tree index on `customer(email)`.	_____ / 2
Concurrency: multiple transactions access the database at the same time; correctness requires isolation. Example: two users update the same account balance.	_____ / 2
Join: combines rows from two relations based on a condition. Example: `orders.customer_id = customers.customer_id`.	_____ / 2
Aggregation: summarizes multiple rows into one result. Example: `COUNT(*)`, `SUM(amount)`, grouped by customer.	_____ / 2
Denormalization: intentional redundancy to improve read performance or simplify queries. Example: storing `order_total` in `orders` although it can be computed.	_____ / 2
Tasks B — Multiple Choice: True or False	_____ / **0**
False	_____ / 2
False	_____ / 2
False	_____ / 2
True	_____ / 2
False	_____ / 2
False	_____ / 2
True	_____ / 2
True	_____ / 2
Tasks C — Normalization and SQL Constraints	_____ / **34**
Accept any three correct dependencies, for example: `customer_id -> customer_name, customer_email`; `event_id -> event_title, event_date, venue_id`; `venue_id -> venue_name, venue_city`; `(event_id, ticket_type) -> ticket_price`; `sale_id -> sale_date, customer_id, payment_method`.	_____ / 6
Award up to 3 points per well-explained anomaly. Examples: repeated customer names/emails for every sale; changing a venue city requires many row updates; deleting the last ticket for an event loses event and venue data; cannot insert a venue before an event or sale if using this table only.	_____ / 6
Expected structure: `Customer(customer_id PK, customer_name, customer_email)`, `Venue(venue_id PK, venue_name, venue_city)`, `Event(event_id PK, event_title, event_date, venue_id FK)`, `Sale(sale_id PK, sale_date, customer_id FK, payment_method)`, and `SaleLine(sale_id PK/FK, event_id PK/FK, ticket_type PK, ticket_price, quantity)` or separate `TicketType(event_id PK/FK, ticket_type PK, ticket_price)` plus line table. Award points for correct keys, FKs, decomposition, and removal of transitive/partial dependencies.	_____ / 14
Award points for syntactically plausible SQL, primary keys, foreign key use, and relevant constraints such as `NOT NULL`, `UNIQUE(customer_email)`, `quantity > 0`, or `ticket_price >= 0`.	_____ / 8
Tasks D — Query Writing	_____ / **22**
Example: `SELECT DISTINCT c.name, c.email FROM Customer c JOIN OrderHeader oh ON oh.customer_id = c.customer_id WHERE oh.status = 'shipped';`	_____ / 6
Example: `SELECT p.category, SUM(oi.quantity * oi.unit_price) AS revenue FROM Product p JOIN OrderItem oi ON oi.product_id = p.product_id GROUP BY p.category;`	_____ / 8
Example: `SELECT c.* FROM Customer c LEFT JOIN OrderHeader oh ON oh.customer_id = c.customer_id WHERE oh.order_id IS NULL;` Alternative with `NOT EXISTS` is also correct.	_____ / 8
Tasks E — Optimized Design Task: Hospital Management System	_____ / **44**
Expected entities: Department, Doctor, Patient, Admission, Treatment. Relationships: Department 1:n Doctor; Patient 1:n Admission; Doctor 1:n Admission as attending doctor; Admission 1:n Treatment; Doctor 1:n Treatment as performing doctor. Award checklist points as specified.	_____ / 12
Expected model: `Department(department_id PK, name UNIQUE)`, `Doctor(doctor_id PK, department_id FK, name, license_no UNIQUE)`, `Patient(patient_id PK, name, date_of_birth, insurance_no UNIQUE NULL)`, `Admission(admission_id PK, patient_id FK, attending_doctor_id FK, admission_date, discharge_date)`, `Treatment(treatment_id PK,	_____ / 14

`admission_id FK, doctor_id FK, treatment_time, treatment_type, notes)`. Equivalent normalized designs are acceptable.	
Admission DDL should reference Patient and Doctor and ensure admission date exists; discharge date should be nullable and should not be before admission date if present. Treatment DDL should reference Admission and Doctor and require timestamp/type. PostgreSQL syntax should be plausible.	_____ / 10
Example: `SELECT p.name AS patient_name, d.name AS doctor_name, dep.name AS department_name FROM Admission a JOIN Patient p ON p.patient_id = a.patient_id JOIN Doctor d ON d.doctor_id = a.attending_doctor_id JOIN Department dep ON dep.department_id = d.department_id WHERE a.discharge_date IS NULL;`	_____ / 8
	_____ / 100 + 0 P.